

Month 2: Purple Team Batch 01

Scripting, Automation & Web Security

Week 08: Cross-Site Scripting (XSS) & Client-Side Attacks

Instructor: **Muhammad Saad**

What is Cross-Site Scripting (XSS)?

XSS stands for **Cross-Site Scripting**. It is a **web security vulnerability** that allows an attacker to inject malicious scripts (usually JavaScript) into a website that other users visit.

When a website does **not properly check or sanitize user input**, an attacker can insert malicious code. That code then runs in the **browser of other users**, not on the attacker's computer.

Example

Imagine a comment box on a website:

Normal user comment:

- This article is helpful!

Attacker comment:

- `<script>alert("You are hacked!")</script>`

If the website does not filter this code, the script runs in every visitor's browser who views the comment.

Stored XSS (Persistent XSS)

Stored XSS happens when the **malicious script is permanently saved on the target website's server** (database, comment section, forum post, user profile, etc.).

How it works

- Attacker submits a malicious script (e.g., in a comment box)
- The website **stores it in the database**
- Every user who views that page unknowingly runs the script in their browser

A comment like:

- `<script>stealCookies()</script>`

If stored, every visitor who loads the page executes it.

Reflected XSS

Reflected XSS happens when the malicious script is **not stored**, but instead is **immediately reflected back by the server in the response**.

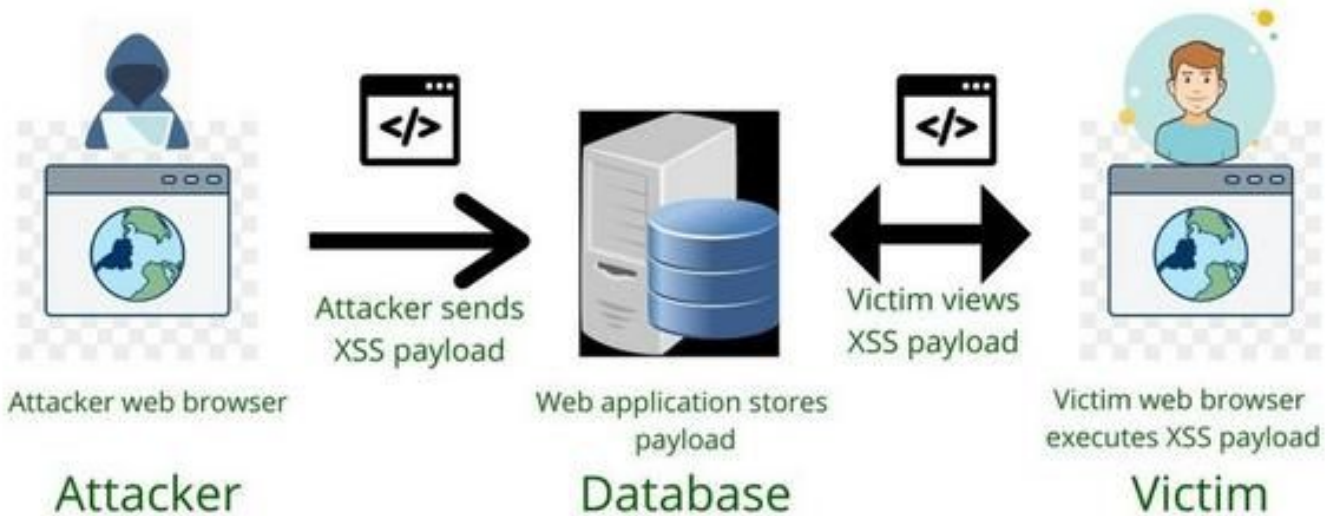
How it works

- Attacker sends a malicious link to a victim
- Victim clicks the link
- The server includes the script in the response page
- The victim's browser executes it

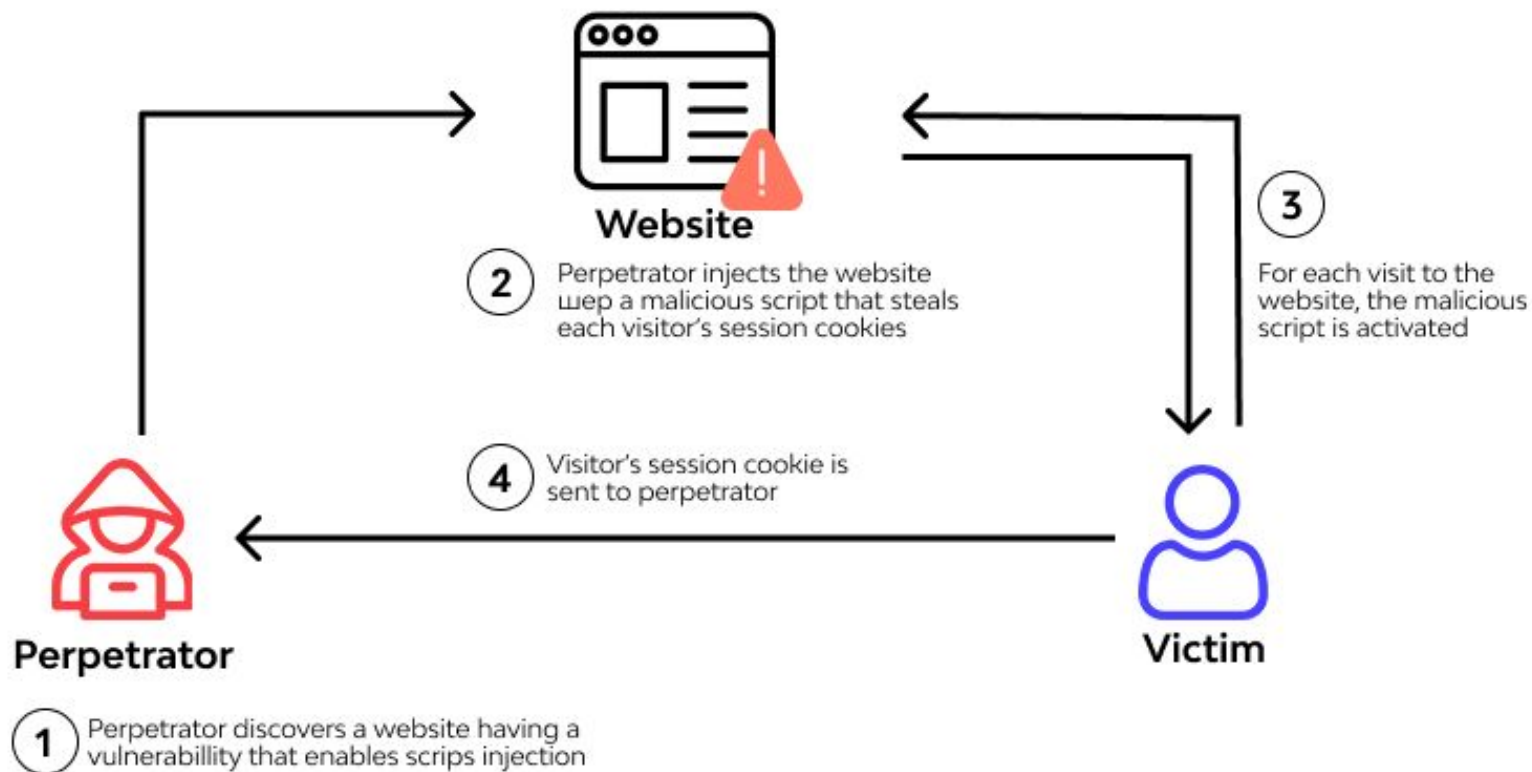
Example URL

- `https://example.com/search?q=<script>alert('XSS')</script>`

If the website directly outputs the `q` parameter without sanitization, the script runs.



Reflected cross-site scripting



What is DOM-based XSS ?

DOM-based XSS is a type of **Cross-Site Scripting** where the vulnerability exists **entirely in the browser (client-side)**, not on the server.

It happens when a website's JavaScript takes **user input and directly modifies the page (DOM) without proper sanitization**, allowing malicious code to run.

DOM stands for **Document Object Model** — it is how a browser represents a webpage so JavaScript can interact with it.

In DOM-based XSS:

- The server does NOT necessarily return malicious content
- The attack happens when **client-side JavaScript processes unsafe input**
- The script is executed by modifying the DOM in the browser

How it works

1. User input is taken from:
 - URL (`window.location`)
 - Fragment (`#value`)
 - Local storage / input fields
2. JavaScript inserts it into the page using unsafe methods like:
 - `innerHTML`
 - `document.write()`
 - `eval()` (very dangerous)
3. Malicious script executes in the browser

Simple Example

Vulnerable JavaScript:

```
document.getElementById("output").innerHTML =  
    location.hash.substring(1);
```

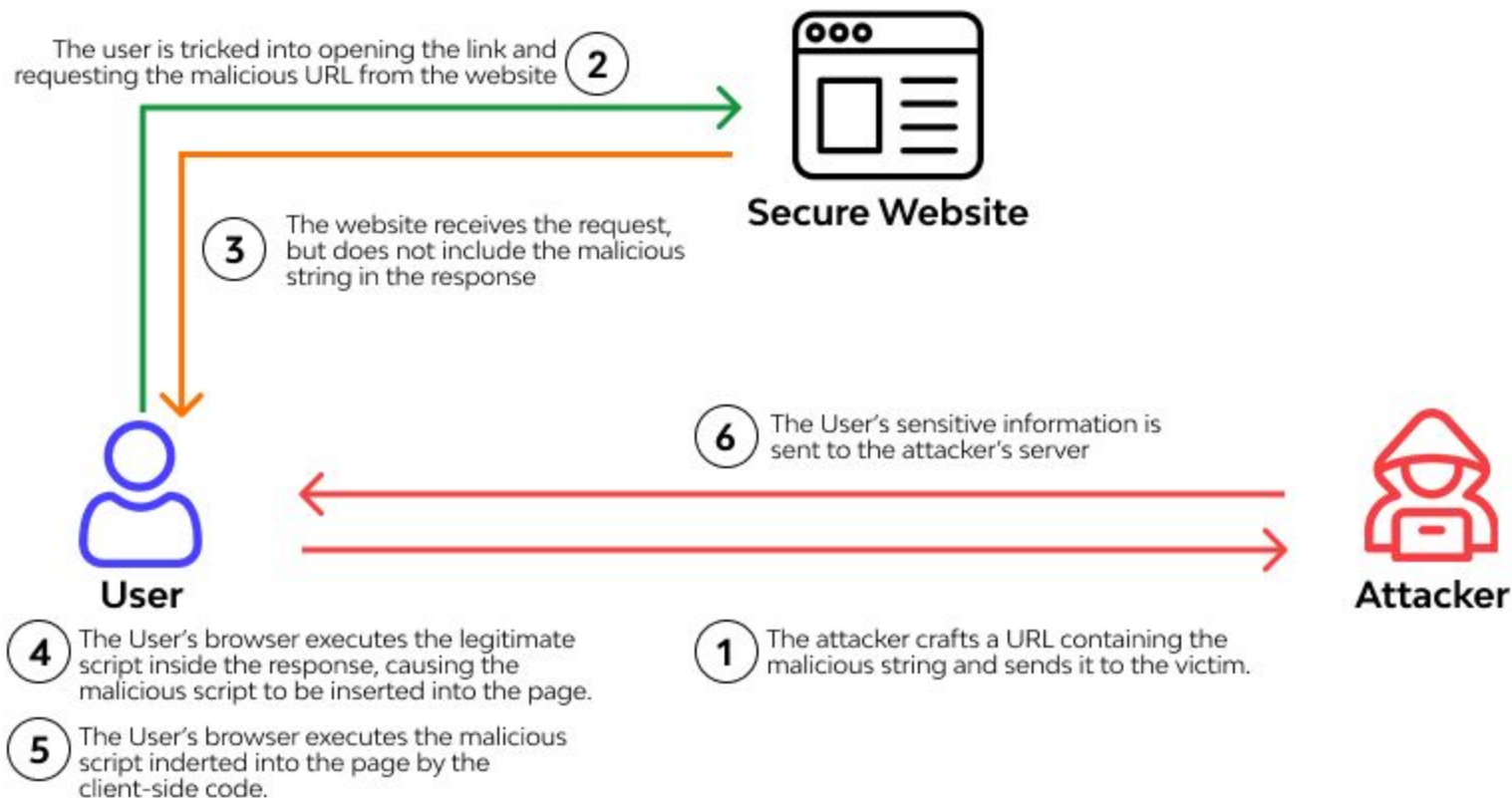
Malicious URL:

- [https://example.com/page#<script>alert\('Hacked'\)</script>](https://example.com/page#<script>alert('Hacked')</script>)

What happens:

- The browser reads `location.hash`
- Inserts it into the page using `innerHTML`
- The script runs in the victim's browser

DOM-based cross-site scripting



What is CSRF?

CSRF stands for **Cross-Site Request Forgery**. It's a type of web attack, but the idea is actually pretty simple.

Imagine this:

- You're logged into your bank account on one tab.
- Without logging out, you visit another (malicious) website.
- That website secretly sends a request to your bank **using your already logged-in session**.

Because your browser is still authenticated, the bank thinks **you** made the request, CSRF tricks your browser into doing something **you didn't intend**, but using your identity.

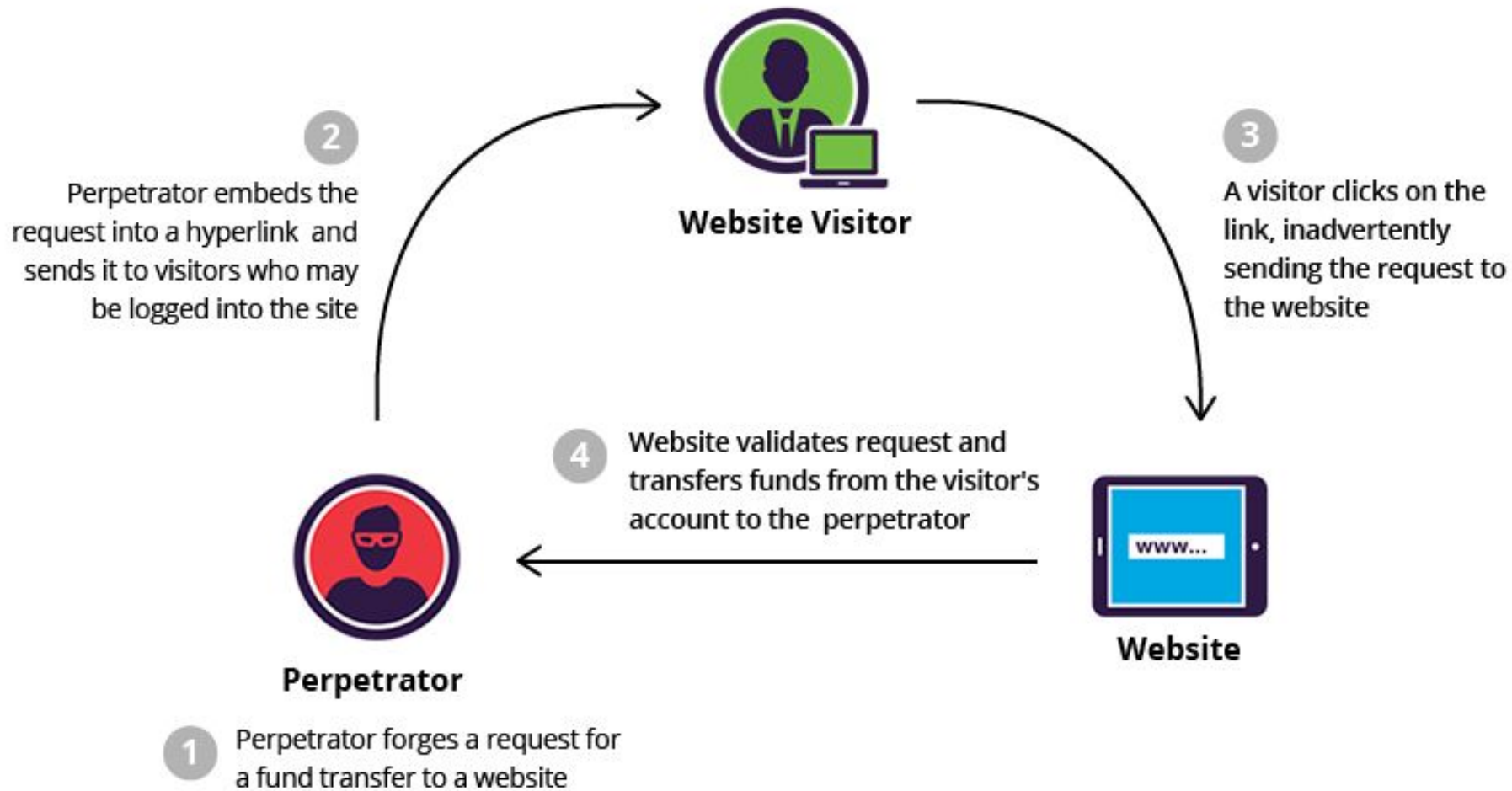
Simple example:

Let's say your bank has a link like:

transfer money to account X

A hacker could hide that request inside a fake website. When you visit it, your browser might automatically:

- send the request
- and transfer money 🤖



Best of Luck for the Week 8 Tasks

Instructor: **Muhammad Saad**